Sequential Sentence Classification in Medical Abstracts

Manual de instalación y ejecución del tablero

Maestría en Inteligencia Artificial – Universidad de los Andes

Natalia Moncada Suarez - 202423306
Santiago Ballesteros Pastrán - 201729106
Cristian Alexis Murillo Martínez - 202325073
Juan Pablo Perez Atehortua - 202424063
Edwin Alexander Cifuentes Bastidas - 202426221

Marzo 2026

1. Introducción

El presente documento describe el proceso de instalación y ejecución del tablero interactivo desarrollado para el proyecto MAIA – PubMed Microproyecto 1. Este tablero permite visualizar la clasificación automática de oraciones de abstracts médicos utilizando un modelo basado en SciBERT entrenado sobre el dataset PubMed-RCT20k.

El sistema está compuesto por tres componentes principales:

- Un modelo de lenguaje basado en SciBERT ajustado para clasificación de oraciones.
- Una API backend implementada con FastAPI que expone el servicio de inferencia.
- Una interfaz web que permite ingresar abstracts médicos y visualizar su clasificación.

El objetivo de este manual es permitir que cualquier usuario pueda instalar y ejecutar el sistema localmente siguiendo una serie de pasos sencillos, por tanto, se tienen dos rutas de instalación para adaptarse a distintos entornos:

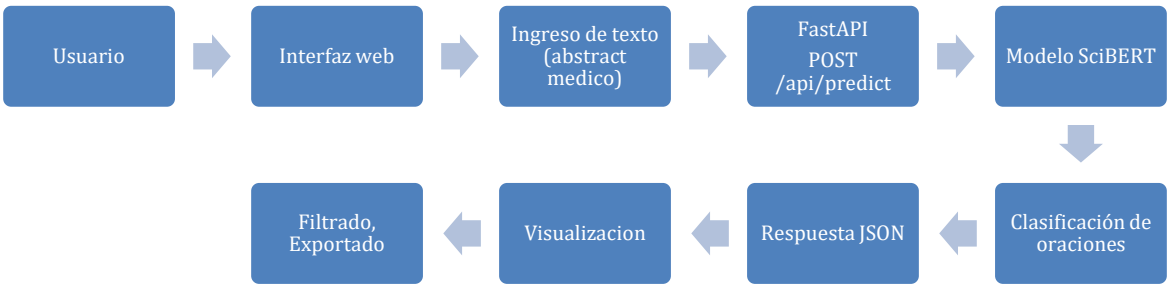
- Ruta A — Docker: despliegue en contenedor, recomendada para producción y equipos de trabajo.
- Ruta B — Sin Docker: ejecución directa con entorno virtual Python, recomendada para desarrollo y depuración.

2. Arquitectura general del sistema

La arquitectura del sistema se compone de tres capas principales que se comunican entre sí de forma secuencial:

Componente	Tecnología	Función
Frontend (SPA)	HTML + JavaScript	Interfaz web: entrada de texto, resaltado de oraciones, filtros, exportación
Backend (API)	FastAPI + Uvicorn	Segmentación NLTK, clasificación SciBERT, rate limiting, seguridad
Modelo de lenguaje	SciBERT (BertForSequenceClassification)	Clasificación en 5 categorías IMRAD (419 MB, SafeTensors)

Flujo de funcionamiento:



Tablero en producción disponible en: http://34.231.109.141:8080/ui/maia-pubmed-microproyecto1.html

El modelo dispone de un fallback automático: si el directorio del modelo de ML no existe o falla la carga, el backend opera en modo heurística determinista con fines de depuración. La interfaz refleja el modo activo mediante el indicador de estado en la barra superior (punto verde = SciBERT activo, punto amarillo = heurística, punto rojo = error de conexión con backend).

3. Requisitos del sistema

Antes de ejecutar el sistema es necesario contar con las siguientes herramientas instaladas:

Herramienta	Versión mínima	Verificar con
Python	3.10	`python --version`
pip	24.0	`pip --version`
Git	2.x	`git --version`
Docker Engine	24.0 (solo Ruta A)	`docker --version`
Docker Compose	2.20 (solo Ruta A)	`docker compose version`

RAM recomendada: mínimo 3 GB disponibles para cargar el modelo SciBERT en CPU.
Disco: ~5 GB libres (419 MB modelo + ~4.5 GB dependencias Python/Docker).
Conexión a internet requerida en los pasos de descarga del modelo y dependencias.

4. Ruta A — Instalación con Docker (recomendada)

Esta ruta empaqueta el backend, el frontend y las dependencias en un único contenedor aislado. Recomendada para despliegue en servidores y entornos reproducibles.

Paso 1 — Clonar el repositorio

```
git clone https://github.com/pabloperez2/maia-pubmed-microproyecto1.git
cd maia-pubmed-microproyecto1
cd tablero
```

Paso 2 — Descargar el modelo desde Google Drive

Los 6 archivos del modelo no están en el repositorio Git (superan el límite de 100 MB de GitHub). Se descargan con el script incluido:

```
python download_model.py
```

El script realiza automáticamente:

- Instala o actualiza gdown si no está disponible.

- Descarga los 6 archivos provenientes de un repositorio publico controlado por el equipo de desarrollo ubicado en Google Drive[1] que serán descargados a la ruta: backend/app/model/scibert_pubmed/.
- Valida la integridad de cada archivo con suma MD5.
- Si todos los archivos ya existen y son válidos, omite la descarga.

Archivos descargados:

Archivo	Tamaño	Descripción
`model.safetensors`	419 MB	Pesos del modelo fine-tuned
`tokenizer.json`	696 KB	Vocabulario SciBERT (31 116 tokens científicos)
`tokenizer_config.json`	~1 KB	Configuración del tokenizador
`config.json`	~1 KB	Arquitectura del modelo
`label_meta.json`	~1 KB	Mapeo label2id / id2label
`training_args.bin`	~5 KB	Argumentos de entrenamiento

```
ubuntu@ip-172-31-79-83: ~/n
.5 nvidia-nvtx-cu12-12.8.90 packaging-26.0 psutil-7.2.2 pydantic-2.12.5 pydantic-core-2.41.5 pygments-2.19
.2 python-dotenv-1.2.2 pyyaml-6.0.3 regex-2026.2.28 rich-14.3.3 safetensors-0.7.0 setuptools-82.0.1 shelli
ngham-1.5.4 starlette-0.52.1 sympy-1.14.0 tokenizers-0.22.2 torch-2.10.0 tqdm-4.67.3 transformers-5.3.0 tr
iton-3.6.0 typer-0.24.1 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.41.0 uvloop-0.22.1 watc
hfiles-1.1.1 websockets-16.0
(.venv) ubuntu@ip-172-31-79-83:~/maia-pubmed-microproyecto1/tablero$ python3 download_model.py
Instalando/actualizando gdown...
✓ gdown listo.

Descargando modelo SciBERT completo desde Google Drive...
  FOLDER_ID : 1ue3EECOilm3U11mV3xctRNDDung25Mg0
  Destino   : /home/ubuntu/maia-pubmed-microproyecto1/tablero/backend/app/model/scibert_pubmed

Retrieving folder contents
Processing file 1UDdDX9rLpbRw0a3IRG_E0eLdf-yzUtKv config.json
Processing file 11b3ilfhWFeT8U2qSajBGb756gpyoX_uP label_meta.json
Processing file 1AX7p0c4K4m3k2xHos76joVh2dy__Yyk7 model.safetensors
Processing file 1VEOwk6n3r2ETYQdj2MrQkK6EeT3qBESV tokenizer_config.json
Processing file 1yc9HKz92kQ_-vf4rdTMk4lKKqKjSyo2A tokenizer.json
Processing file 1W6xO9yXtvrLKZJxJtU8s4mfUovW_toka training_args.bin
Retrieving folder contents completed
Building directory structure
Building directory structure completed
Downloading...
From: https://drive.google.com/uc?id=1UDdDX9rLpbRw0a3IRG_E0eLdf-yzUtKv
To: /home/ubuntu/maia-pubmed-microproyecto1/tablero/backend/app/model/scibert_pubmed/config.json
100%|| 987/987 [00:00<00:00, 3.97MB/s]
Downloading...
From: https://drive.google.com/uc?id=11b3ilfhWFeT8U2qSajBGb756gpyoX_uP
To: /home/ubuntu/maia-pubmed-microproyecto1/tablero/backend/app/model/scibert_pubmed/label_meta.json
100%|| 319/319 [00:00<00:00, 1.58MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1AX7p0c4K4m3k2xHos76joVh2dy__Yyk7
From (redirected): https://drive.google.com/uc?id=1AX7p0c4K4m3k2xHos76joVh2dy__Yyk7&confirm=t&uuid=2a694ec
3-8ab7-419a-8318-bf8eb8c834b8
To: /home/ubuntu/maia-pubmed-microproyecto1/tablero/backend/app/model/scibert_pubmed/model.safetensors
100%|| 440M/440M [00:03<00:00, 138MB/s]
Downloading...
From: https://drive.google.com/uc?id=1VEOwk6n3r2ETYQdj2MrQkK6EeT3qBESV
To: /home/ubuntu/maia-pubmed-microproyecto1/tablero/backend/app/model/scibert_pubmed/tokenizer_config.json
100%|| 351/351 [00:00<00:00, 1.77MB/s]
Downloading...
From: https://drive.google.com/uc?id=1yc9HKz92kQ_-vf4rdTMk4lKKqKjSyo2A
```

[1] https://drive.google.com/drive/folders/1ue3EECOilm3U11mV3xctRNDDung25Mg0

Salida esperada al finalizar correctamente:

```
Validando integridad de archivos descargados...
  model.safetensors              calculando MD5... ✓  419.0 MB  md5 ✓
  tokenizer.json                 calculando MD5... ✓  0.7 MB    md5 ✓
  tokenizer_config.json          calculando MD5... ✓  0.0 MB    md5 ✓
  config.json                    calculando MD5... ✓  0.0 MB    md5 ✓
  label_meta.json                calculando MD5... ✓  0.0 MB    md5 ✓
  training_args.bin              calculando MD5... ✓  0.0 MB    md5 ✓

✓ Modelo completo e integro en: .../backend/app/model/scibert_pubmed

Ahora puede construir el contenedor:
  docker compose build
  docker compose up -d
```

Paso 3 — Instalar Docker

Windows

1. Descargar Docker Desktop desde https://www.docker.com/products/docker-desktop
2. Ejecutar el instalador y reiniciar el sistema.
3. Abrir Docker Desktop y esperar a que el motor arranque (ícono en la barra de tareas).
4. Verificar en terminal:

```
docker --version
docker compose version
```

macOS:

```
# Con Homebrew:
brew install --cask docker
# Abrir Docker Desktop desde Aplicaciones y esperar a que arranque
docker --version
docker compose version
```

Linux (Ubuntu / Debian):

```
# Eliminar versiones previas
sudo apt-get remove docker docker-engine docker.io containerd runc

# Instalar Docker Engine
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor \
     -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io \
     docker-buildx-plugin docker-compose-plugin

# Agregar usuario al grupo docker (evita usar sudo)
sudo usermod -aG docker $USER
newgrp docker

# Verificar
docker --version
docker compose version
docker run hello-world
```

Paso 4 — Construir la imagen Docker

Desde la raíz de la carpeta tablero/ (donde se encuentra el Dockerfile):

```
docker compose build
```

La primera vez descarga e instala las dependencias (~950 MB: torch CPU, transformers, etc.).

Las capas quedan en caché — builds posteriores son mucho más rápidos.

Paso 5 — Levantar el contenedor

```
docker compose up -d

# Ver logs en tiempo real:
docker compose logs -f
```

El contenedor tarda 30-60 segundos en estar listo la primera vez (carga SciBERT en memoria).
El healthcheck aparece como 'starting' durante ese período: es comportamiento normal.

Paso 6 — Verificar el despliegue

Comprobar el estado del contenedor (debe aparecer healthy):

```
docker compose ps
```

Consultar el endpoint de salud:

```
curl http://localhost:8080/health
```

Respuesta esperada:

```
{ "status": "ok",
  "model_version": "allenai/scibert_scivocab_cased-v0.0.1",
  "model_backend": "scibert_finetuned",
  "uptime_s": 73.2,
  "max_text_length": 8000,
  "max_rpm": 20
}
```

model_backend	Significado
scibert_finetuned	Modelo cargado correctamente
heuristic_fallback	Modelo no encontrado — verificar download_model.py sin errores

Abrir la interfaz web:

```
http://localhost:8080/
```

Detener el contenedor

```
# Detener (conserva la imagen)
docker compose down

# Detener y eliminar la imagen construida
docker compose down --rmi local
```

5. Ruta B — Instalación sin Docker (desarrollo)

Útil para desarrollo activo y depuración. Todos los comandos se ejecutan desde la raíz de la carpeta tablero/.

Paso 1 — Clonar el repositorio

```
git clone https://github.com/pabloperez2/maia-pubmed-microproyecto1.git
cd maia-pubmed-microproyecto1
cd tablero
```

Paso 2 — Instalar el módulo venv (Ubuntu/Debian)

En sistemas Ubuntu o Debian puede que el módulo venv no esté instalado por defecto. Ejecutar:

```
sudo apt install python3.12-venv
```

En Windows y macOS el módulo venv ya viene incluido con Python. Omitir este paso.

Paso 3 — Crear y activar el entorno virtual

Linux / macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows — CMD:

```
python3 -m venv .venv
.venv\Scripts\activate.bat
```

Windows — PowerShell:

```
python3 -m venv .venv
.venv\Scripts\Activate.ps1
```

Si PowerShell bloquea la ejecución de scripts, ejecutar primero:Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Paso 4 — Instalar dependencias

Con el venv activo, desde la raíz de tablero/:

```
pip install -r backend/requirements.txt
```

Principales dependencias instaladas:

Paquete	Función
FastAPI + Uvicorn	Servidor web ASGI y API REST
torch (CPU)	Backend de inferencia para el modelo SciBERT
transformers	Pipeline de clasificación de texto (HuggingFace)
safetensors	Carga eficiente y segura de los pesos del modelo
nltk	Tokenización robusta de oraciones (abreviaturas médicas)
pydantic	Validación de datos de entrada / salida de la API
gdown	Descarga del modelo desde Google Drive

Paso 5 — Descargar el modelo desde Google Drive

Los 6 archivos del modelo no están en el repositorio Git (superan el límite de 100 MB de GitHub). Ejecutar el script de descarga una única vez:

```
python3 download_model.py
```

Si todos los archivos ya existen y son válidos, el script omite la descarga.

El script instala gdown automáticamente, descarga los archivos y valida su integridad con MD5. Ver salida esperada en la Sección 4 — Paso 2.

Paso 6 — Arrancar el backend

Con el venv activo, desde la raíz de tablero/:

Linux / macOS:

```
MODEL_DIR=./backend/app/model/scibert_pubmed \
MODEL_VERSION=allenai/scibert_scivocab_cased-v0.0.1 \
MAX_TEXT_LENGTH=8000 \
MAX_RPM=20 \
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

Windows — CMD:

```
set MODEL_DIR=.\backend\app\model\scibert_pubmed
set MODEL_VERSION=allenai/scibert_scivocab_cased-v0.0.1
set MAX_TEXT_LENGTH=8000
set MAX_RPM=20
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

Windows — PowerShell

```
$env:MODEL_DIR=".\backend\app\model\scibert_pubmed"
$env:MODEL_VERSION="allenai/scibert_scivocab_cased-v0.0.1"
$env:MAX_TEXT_LENGTH="8000"
$env:MAX_RPM="20"
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

El modelo tarda 30-60 segundos en cargar la primera vez. La flag --reload recarga automáticamente el servidor al guardar cambios (solo desarrollo).

Paso 7 — Verificar

```
curl http://localhost:8080/health
```

Revisar el campo model_backend en la respuesta: debe ser scibert_finetuned. Si muestra heuristic_fallback verificar que download_model.py completó correctamente.

Abrir la interfaz web:

```
http://localhost:8080/
```

Arranque mínimo

Si solo se quiere probar la interfaz sin cargar SciBERT, omitir las variables de entorno:

Linux / macOS:

```
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

Windows — CMD:

```
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

Windows — PowerShell:

```
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

6. Variables de entorno

Variable	Valor por defecto	Descripción
`MODEL_VERSION`	allenai/scibert_scivocab_cased-v0.0.1	Versión mostrada en la UI y en /health
`MODEL_DIR`	./backend/app/model/scibert_pubmed	Ruta al directorio del modelo
`MAX_TEXT_LENGTH`	8000	Límite de caracteres por solicitud
`MAX_RPM`	20	Máximo de solicitudes por minuto por IP
`FRONTEND_DIR`	/app/frontend	Ruta a los archivos estáticos del SPA
`ALLOWED_ORIGINS`	http://localhost:8080,...	Orígenes CORS permitidos (separados por coma)

7. Referencia de la API

POST /api/predict

Clasifica cada oración del abstract recibido y devuelve etiqueta (Background, Objective, Methods, Results, Conclusions) y puntuación de confianza.

```
# Cuerpo de la solicitud (JSON):
{ "text": "Pegue aqui el abstract completo en ingles..." }

# Respuesta exitosa (200):
{
  "model_version": "allenai/scibert_scivocab_cased-v0.0.1",
  "sentences": [
    {
      "index": 1,
      "start": 0,
      "end": 95,
      "sentence": "This study evaluates...",
      "label": "Objective",
      "confidence": 0.934
    }
  ]
```

```
}
```

Códigos de respuesta HTTP

Código HTTP	Significado
200	Clasificación exitosa
400	Texto vacío o contiene patrones de inyección XSS/SQLi
413	Texto supera MAX_TEXT_LENGTH caracteres
422	Error de validación del cuerpo JSON
429	Rate limit excedido — reintentar en 60 s

GET /health

Devuelve el estado del sistema, versión del modelo y configuración activa. Usado por el indicador de la barra superior del tablero (polling cada 45 s).

```
{ "status": "ok",
  "model_version": "allenai/scibert_scivocab_cased-v0.0.1",
  "model_backend": "scibert_finetuned",
  "uptime_s": 120.4,
  "max_text_length": 8000,
  "max_rpm": 20
}
```

8. Modelo

Campo	Valor
Modelo base	allenai/scibert_scivocab_cased
Dataset de fine-tune	armanc/pubmed-rct20k (PubMed-RCT20k)
Versión fine-tune	v0.0.1 (etiqueta interna del equipo, no una versión oficial de HuggingFace)
Arquitectura	BertForSequenceClassification (BERT-base, 12 capas)
Vocabulario	31 116 tokens científicos (SciBERT cased)
Formato de pesos	SafeTensors — model.safetensors (419 MB)
Clases	5 — Background, Objective, Methods, Results, Conclusions

9. Seguridad

El sistema implementa varias capas de protección tanto en el backend como en el frontend:

Mecanismo	Descripción
Validación XSS / SQLi	El backend rechaza texto con patrones de inyección HTML, JavaScript o SQL
Rate limiting	Máximo 20 solicitudes por minuto por IP en /api/predict (configurable con MAX_RPM)
Cabeceras de seguridad	X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Content-Security-Policy
CORS restringido	Solo se aceptan orígenes listados en ALLOWED_ORIGINS
Límite de payload	Textos mayores a MAX_TEXT_LENGTH caracteres son rechazados con HTTP 413
Normalización Unicode	Entradas normalizadas a NFC para evitar bypass por homoglifos

10. Solución de problemas comunes

Síntoma	Causa probable	Solución
model_backend = heuristic_fallback	MODEL_DIR vacío o modelo no descargado	Ejecutar download_model.py y verificar que todos los archivos pasaron MD5
Contenedor en starting > 2 min	RAM insuficiente o timeout de carga	Verificar RAM disponible (min. 4 GB); revisar docker compose logs -f
Error 429 en /api/predict	Rate limit excedido (20 RPM)	Esperar 60 s o aumentar MAX_RPM en docker-compose.yml
Error 413 Payload demasiado grande	Texto supera 8 000 caracteres	Reducir el texto o aumentar MAX_TEXT_LENGTH
Puerto 8080 ocupado	Otro proceso usa el puerto	Cambiar a '9090:8080' en docker-compose.yml
curl: (7) Failed to connect	Backend no arrangó o firewall/security group	Verificar docker compose ps y regla inbound TCP 8080
MD5 no coincide	Descarga incompleta o archivo corrupto	El script elimina el archivo y permite reintentar; volver a ejecutar
ImportError: transformers	Dependencias no instaladas o venv no activo	pip install -r backend/requirements.txt con el venv activo
Error CORS en el navegador	Origen no en ALLOWED_ORIGINS	Añadir el origen o usar ALLOWED_ORIGINS=*

11. Resumen rápido de comandos

Ruta A — Docker

```
git clone https://github.com/pabloperez2/maia-pubmed-microproyecto1.git
cd maia-pubmed-microproyecto1/tablero
python download_model.py
docker compose build
docker compose up -d
curl http://localhost:8080/health
# Abrir en navegador: http://localhost:8080/
```

Ruta B — Sin Docker (Linux / macOS)

```
git clone https://github.com/pabloperez2/maia-pubmed-microproyecto1.git
cd maia-pubmed-microproyecto1/tablero
sudo apt install python3.12-venv          # Ubuntu: instalar modulo venv
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt
python3 download_model.py
MODEL_DIR=./backend/app/model/scibert_pubmed \
MODEL_VERSION=allenai/scibert_scivocab_cased-v0.0.1 \
MAX_TEXT_LENGTH=8000 MAX_RPM=20 \
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

Ruta B — Sin Docker (Windows PowerShell)

```
git clone https://github.com/pabloperez2/maia-pubmed-microproyecto1.git
cd maia-pubmed-microproyecto1\tablero
python3 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
python3 download_model.py
$env:MODEL_DIR=".\backend\app\model\scibert_pubmed"
$env:MODEL_VERSION="allenai/scibert_scivocab_cased-v0.0.1"
$env:MAX_TEXT_LENGTH="8000"; $env:MAX_RPM="20"
uvicorn backend.app.main:app --host 0.0.0.0 --port 8080 --reload
```

12. Conclusión

El tablero de MAIA — PubMed Microproyecto 1 ofrece dos rutas de instalación que se adaptan a distintos contextos: Docker para entornos reproducibles y de producción, y entorno virtual Python para desarrollo iterativo.

La arquitectura modular del sistema — frontend estático servido por FastAPI, modelo SciBERT cargado en memoria y fallback heurístico automático — facilita tanto el despliegue como la extensión futura, por ejemplo, la incorporación de nuevos modelos de clasificación o la integración con sistemas clínicos externos.